OPERATING SYSTEMS LAB · PRACTICAL SECTION 3

File Operations, Redirection & Command Chaining

Practical lab guide: command chaining (; vs &&), file lifecycle (CRUD), streams (> vs >>), and text utilities.

bash v5.2

```
user@linux:~$ mkdir -v os_lab3 && cd os_lab3
mkdir: created directory 'os_lab3'
user@linux:~/os_lab3$ touch "lab notes.txt" && echo "OS Lab 3" > "lab notes.txt"
user@linux:~/os_lab3$ cat "lab notes.txt"
OS Lab 3
user@linux:~/os_lab3$ cp -r . backup/ ; ls backup/
'lab notes.txt'
```

Today: What We Will Practice in Lab 3

01

Chaining (; & &&)

• Sequential execution with ';' vs conditional chaining with '&&', plus multi-level hops.

02

File Creation

• Creating 0-byte files with 'touch', absolute paths, and escaping spaces in names.

03

Copy & Move

• Duplicating files and directories with 'cp -r', and relocating/renaming with 'mv'.

04

Safe Deletion

• Removing empty folders with 'rmdir', files with 'rm', and safe recursive wipes 'rm -ri'.

05

Text & Streams

• Understanding 'echo' & 'cat', followed by output redirection ('>' overwrite vs '>>' append).

Command Chaining: Semicolon (;) vs. Logical AND (&&)

ALWAYS RUNS NEXT

Semicolon (;)

Unconditional Sequential Execution

- Runs command2 regardless of whether command1 succeeds or fails.
- Use when tasks are completely independent of each other.

EXAMPLES

```
$ ls ; date ; pwd
# If 'ls' fails, 'date' and 'pwd' still execute!
```

RUNS ONLY ON SUCCESS

Logical AND (&&)

Conditional Safe Chaining

- Runs command2 ONLY if command1 succeeds (Exit Code 0).
- If command1 fails, execution stops immediately.
- Crucial when command2 depends on command1!

EXAMPLES

```
$ mkdir os_lab3 && cd os_lab3
# If mkdir fails, terminal will NOT enter a nonexistent folder!
```

Multi-Level Jumps & Directory Navigation

PARENT JUMPS

Multi-Level Hop: cd ../..

Climbing the Tree Faster

- '..' moves up exactly one level (parent).
- '../..' chains two parent jumps in one step.
- '../../..' climbs three levels up.
- Saves time compared to running 'cd ..' multiple times.

EXAMPLES

```
$ pwd
/home/username/Documents/OS
$ cd ../..
$ pwd
/home/username
```

SHORTCUT SYMBOLS

Quick Navigation Summary

Symbols Every Student Must Know

- cd ~ (or cd) -> Jumps to Home directory (/home/username)
- cd - -> Jumps back to previous working directory
- cd / -> Jumps to Root directory (top of the file tree)
- cd . -> Current directory (useful in scripts and paths)

EXAMPLES

```
$ cd /var/log
$ cd -          # Returns to previous folder
$ cd ~          # Back to home immediately
```

Creating Files: touch & Target Paths

BASIC USAGE

Single File

Creates empty 0-byte file

- Creates an empty file if it doesn't exist.
- If file already exists: updates last-modified timestamp without erasing content!

EXAMPLES

```
$ touch notes.txt
$ ls -l notes.txt
-rw-r-- 1 user 0 Oct 8 notes.txt
```

MULTIPLE ARGS

Multiple Files

Batch creation in one command

- Pass multiple arguments separated by spaces.
- Creates all files simultaneously.
- Fast setup for labs and projects.

EXAMPLES

```
$ touch file1 file2 file3
$ ls
file1  file2  file3
```

PATH TARGET

Specific Path

Direct creation without cd

- Specify absolute or relative path to create file directly in destination folder.
- No need to move into folder first.

EXAMPLES

```
$ touch ~/Documents/lab3.c
$ touch /tmp/test.log
```

Handling Spaces in File Names: The Space Trap

COMMON MISTAKE

The Problem: Space Delimiter

Why bash splits names by default

- The Shell uses whitespace to separate arguments.
- Typing: touch file 1 creates TWO files: 'file' and '1'.
- To keep spaces inside a single file name, you must escape or quote them.

EXAMPLES

```
# The mistake:
$ touch file 1
$ ls
1  file        # Two files created!
```

THE SOLUTION

The Solutions: Quotes & Escape

3 Valid Ways to Handle Spaces

- 1. Double Quotes: touch "file 1"
- 2. Single Quotes: touch 'file 2'
- 3. Backslash Escape: touch file\ 3
- Pro-Tip: Pressing Tab automatically inserts the backslash escape for you!

EXAMPLES

```
# Correct syntax:
$ touch "file 1" 'file 2' file\ 3
$ ls
'file 1'  'file 2'  'file 3'
```

Copying Files & Directories: cp & cp -r

FILE TO FILE

Copy to New File

Duplicates with new name

- First argument = source file.
- Second argument = target file name.
- Creates independent duplicate.

EXAMPLES

```
$ cp file1 file1_backup
$ ls
file1  file1_backup
```

FILE TO DIRECTORY

Copy into Directory

Preserves file name inside folder

- Can copy single or multiple files.
- Last argument MUST be a directory.
- Files are copied into that folder.

EXAMPLES

```
$ cp file1 Documents/
$ cp f1 f2 f3 lab1/
```

RECURSIVE FLAG

Copy Directory (-r)

Recursive folder duplication

- Directories require '-r' (Recursive).
- Copies folder and all subfolders & files.
- Without -r, cp fails with an error!

EXAMPLES

```
$ cp -r lab1 lab2
# Without -r:
cp: omitting directory 'lab1'
```

Moving & Renaming: The Unified mv Command

RENAME FILE

Rename in Place

Same folder, new name

- In Linux, rename and move are the same operation.
- Source and target are in the same directory = rename.

EXAMPLES

```
$ mv draft.txt report.txt
$ ls
report.txt
```

RELOCATE FILES

Move to Directory

Relocate files across folders

- Can move single or multiple files.
- Last argument must be destination directory.
- Original copies are removed.

EXAMPLES

```
$ mv report.txt Documents/
$ mv f1 f2 f3 lab1/
```

DIRECTORY MOVE

Move / Rename Directory

No recursive flag needed!

- Unlike 'cp', 'mv' does NOT require '-r' to move or rename folders.
- Works instantly on entire directory hierarchies.

EXAMPLES

```
$ mv lab1/ lab2/
$ mv old_project/ ~/backup/
```

File & Directory Deletion: rmdir, rm, and rm -ri

EMPTY ONLY

rmdir (Safe Folders)

Deletes EMPTY directories only

- Safeguard against accidental loss.
- If folder contains ANY files, rmdir safely refuses to delete.

EXAMPLES

```
$ rmdir empty_lab/
# If not empty:
rmdir: Directory not empty
```

PERMANENT

rm (Remove Files)

Permanently deletes files

- WARNING: Linux has NO Recycle Bin!
- Files deleted via terminal are permanently destroyed.
- Deletes single or multiple files.

EXAMPLES

```
$ rm file1
$ rm file2 file3
```

SAFETY FLAG -I

rm -r & rm -ri

Recursive wipe & interactive prompt

- '-r': Recursively wipes folder and everything inside it.
- '-i': Interactive mode: prompts (y/n) before deleting each item. Always use -i in lab!

EXAMPLES

```
$ rm -ri lab2/
rm: remove file 'file1'? y
rm: remove directory? y
```

Before Redirection: Understanding echo & cat

OUTPUT SPEAKER

echo — The Terminal Speaker

Prints text to standard output (stdout)

- Takes input arguments and prints them directly to terminal screen.
- Programming equivalent (C++): std::cout << text << std::endl;
- Used for displaying text messages, checking variable values, and generating output data.

EXAMPLES

```
$ echo Hello OS Lab
Hello OS Lab
$ echo "Section: 3"
Section: 3
```

STREAM READER

cat — The File Reader & Combiner

Reads files and prints to standard output (stdout)

- Short for 'concatenate' (link items together).
- Opens text files and dumps their complete contents onto screen.
- Programming equivalent (C++): ifstream file >> std::cout;
- Used for inspecting files quickly and combining multiple files.

EXAMPLES

```
$ cat /etc/hostname
ubuntu-vm
$ cat notes.txt
Practical section notes
```

The Output Redirection Mental Model: > vs. >>

OVERWRITE ARROW

> (Overwrite / Truncate)

Clears previous contents and writes fresh

- Redirects standard output into a file instead of the screen.
- If file does not exist: creates it.
- If file exists: CLEARS (wipes) all previous content and replaces it.
- Think: 'Overwrite arrow'.

EXAMPLES

```
$ echo "First Line" > file.txt
$ echo "New Line" > file.txt
$ cat file.txt
New Line    # 'First Line' is gone!
```

APPEND ARROW

>> (Append / Preserve)

Adds new text to the bottom

- Redirects standard output to the end of a file.
- If file does not exist: creates it.
- If file exists: ADDS new text to the bottom, keeping all previous content intact.
- Think: 'Append arrow'.

EXAMPLES

```
$ echo "Line 1" > file.txt
$ echo "Line 2" >> file.txt
$ cat file.txt
Line 1
Line 2       # Both lines preserved!
```

Writing Text with echo: Screen vs. Files

DEFAULT STDOUT

Print to Screen

Default stdout behavior

- Prints text strings directly to terminal window.
- No redirection used.

EXAMPLES

```
$ echo hello
hello
$ echo "OS Lab 3"
OS Lab 3
```

OVERWRITE (>)

Overwrite to File (>)

Creates file with initial content

- Takes text and writes it into file.
- Creates the file automatically if missing.
- Wipes old content if file exists.

EXAMPLES

```
$ echo hello > file1
$ cat file1
hello
```

APPEND (>>)

Append to File (>>)

Logs additional lines

- Adds text to the end of the file.
- Useful for logging events and appending data line-by-line.

EXAMPLES

```
$ echo "Line 2" >> file1
$ cat file1
hello
Line 2
```

Reading, Merging & Interactive Input: cat

READ / CONCAT

Display Files

Single & sequential reading

- cat file1: displays single file.
- cat f1 f2: displays multiple files sequentially in order.

EXAMPLES

```
$ cat file1
hello
$ cat file1 file2
hellowelcome
```

MERGE VIA >

Copy / Merge via >

Diverts stream into new file

- cat f1 > f2: copies content of f1 into f2.
- cat f1 f2 > full.txt: merges two files into a third file!

EXAMPLES

```
$ cat f1 f2 > full.txt
$ cat full.txt
hellowelcome
```

INTERACTIVE (> & >>)

Interactive Input (> & >>)

Type notes directly until Ctrl+D

- cat > file: creates / overwrites file from keyboard.
- cat >> file: appends new lines from keyboard without erasing previous content!
- When finished typing, press: Ctrl + D (EOF signal).

EXAMPLES

```
$ cat > notes.txt
Line 1
^D   # (saves file)
$ cat >> notes.txt
Line 2
^D   # (appends line!)
```

Hands-on Practice: Step-by-Step Lab Tasks

1. **Workspace Setup:** Navigate to your home directory, create a folder named 'os_lab3' using conditional chaining (&&), and step inside.
2. **File Creation:** Create two empty files: 'file1.txt' and a file with whitespace in its name: 'my notes.txt'.
3. **Write Initial Content:** Use 'echo' with overwrite redirection (>) to write the line "Linux OS Lab 3" into 'file1.txt'.
4. **Append Additional Line:** Use 'echo' with append redirection (>>) to add "Redirection Mastered" to 'file1.txt' without erasing existing text.
5. **Inspect File Contents:** Display the contents of 'file1.txt' on the terminal screen using 'cat' to verify both lines are present.
6. **Duplicate Backup:** Make an exact duplicate copy of 'file1.txt' named 'backup.txt' inside the current folder.
7. **Interactive Cleanup:** Safely delete 'backup.txt' using the interactive confirmation flag so the shell prompts before deleting.
8. **Tree Climb:** Move two directory levels up in the filesystem hierarchy in a single command.

Self-Discovery Challenge: Research Flags in Terminal

`LS -S & LS -T`

Options for ls

• Investigate what the -S (capital) and -t options do when executed with ls.

`RM -F`

Flag for rm

• Investigate what the -f flag does during deletion, and why pairing it with -r is dangerous.

`CP -U`

Flag for cp

• Investigate the behavior of the -u flag when copying files to a destination.

`CAT -N`

Flag for cat

• Investigate what output modification the -n flag produces when displaying file contents.

SECTION 3 SUMMARY

You Can Now Manage Files & Control Data Streams!

Chaining: `;` (unconditional) | `&&` (conditional on success) | `cd ../..` (multi-hop)

Creation: `touch` (empty file / timestamp) | quotes & escaping for names with spaces

Duplication & Move: `cp` (files) | `cp -r` (recursive directories) | `mv` (rename & move)

Safe Deletion: `rm` (files) | `rmdir` (empty folders) | `rm -ri` (safe interactive wipe)

Text & Streams: `echo` (stdout speaker) | `cat` (stream reader) | `>` (overwrite) | `>>` (append)